

작업일지

2026년 4월 3일 (금)

온라인 강의 플랫폼 (학생용) 프로젝트

[오늘 한 일]

프론트엔드와 NestJS 백엔드 API를 실제로 연동하였다. 기존에 Mock 데이터 기반으로 동작하던 AppContext의 모든 상태 관리 로직을 실제 HTTP 요청으로 교체하였으며, 별도의 API 서비스 레이어를 신규 생성하여 fetch 호출을 중앙화하였다. Vite 개발 서버의 프록시 설정과 NestJS의 CORS 설정을 함께 적용하여 크로스 오리진 이슈 없이 연동이 가능하도록 구성하였다. 또한 루트 package.json의 workspaces 설정을 제거하고 BE/FE 각각 독립 실행 구조로 정리하였다.

[완료한 작업]

NestJS main.ts에 CORS 허용 설정 추가 (Vite 개발 서버 origin 허용), Vite 개발 서버 프록시 설정 추가 (/api → http://localhost:3000 경로 재작성), FE/src/services/api.ts 신규 생성 — courseApi / userApi / enrollmentApi 모듈로 분리하여 fetch 기반 공통 request 함수 구현, AppContext 전면 리팩터링 — Mock 데이터 및 users 상태 제거, GET /courses 마운트 시 호출로 강의 목록 초기 로드, GET /users/:id/enrollments 현재 사용자 변경 시 수강 목록 자동 갱신, 강의 추가·수정·삭제·수강 신청·수강 취소 모두 async/await 기반 API 호출로 전환, currentUser 상태를 localStorage에 영속화하여 새로고침 후에도 유지되도록 처리, 각 페이지(CourseDetailPage, CourseFormPage, UserRegisterPage, EnrollmentListPage) 이벤트 핸들러를 async 함수로 변환 및 에러 핸들링 추가, currentUser가 null인 경우 Navbar, EnrollmentListPage, CourseDetailPage에서 안전하게 처리, 루트 package.json workspaces/scripts 제거 및 루트 node_modules 삭제 — BE/FE 독립 실행 구조로 정리.

[어려웠던 점]

AppContext의 메서드 시그니처를 동기에서 비동기로 전환하면서 각 페이지 핸들러도 함께 수정해야 했는데, 기존 코드에서 반환값을 기다리지 않고 바로 `alert.navigate`를 호출하던 패턴이 많아 누락 없이 전부 수정하는 데 주의가 필요했다. 또한 `currentUser`를 `null`로 허용하는 타입 변경으로 인해 기존 코드에서 `currentUser`를 직접 참조하던 모든 지점에 `null` 체크를 추가해야 했다. 수강 목록 페이지의 경우 기존에 Context의 `courses` 배열에서 `find`로 강의를 찾던 방식을, 백엔드 `enrollments` 응답에 `course` 관계 (`relations: ['course']`)가 포함되어 있어 이를 우선 사용하고 없을 경우 `courses` 배열에서 `fallback`하는 방식으로 개선하였다. 모노레포 구조에서 루트 `node_modules`에 `class-validator`가 없어 NestJS `ValidationPipe` 경고가 발생하였고, `workspaces`를 제거하고 BE/FE를 각각 독립 실행하는 방식으로 해결하였다.